

SQL & DATABASES · DAY 5 OF 10

Joining tables

INNER JOIN, LEFT JOIN and joining many tables

Friday 9 October 2026

Today

- After today you can:
 - Explain why we need JOIN
 - Join two tables with INNER JOIN ... ON
 - Use short names (aliases) for tables
 - Join three or four tables
 - Use LEFT JOIN to find things that are missing

Today's schedule

Time	What we do
09:00-09:10	Warm-up
09:10-09:50	Theory (slides)
09:50-10:00	Break
10:00-10:55	Code-along, part 1
10:55-11:05	Break
11:05-12:00	Code-along, part 2
12:00-13:00	Lunch
13:00-14:30	Lab, part 1
14:30-14:45	Break
14:45-16:15	Lab, part 2
16:15-17:00	Walkthrough and wrap-up

Our data is in pieces

- Who is customer 1? We have to look in another table
- That's good design... but we want to see it together

order_id	customer_id	order_date
1	1	2026-01-05
2	2	2026-01-12

JOIN = matching puzzle pieces

- JOIN glues rows from two tables together
- ON tells SQL which columns should match
- orders.customer_id = customers.customer_id

```
SELECT orders.order_id, customers.first_name  
FROM orders  
JOIN customers ON orders.customer_id = customers.customer_id;
```

What INNER JOIN gives you

- Only rows that have a match in BOTH tables
- Emma has no orders, so she doesn't appear
- JOIN and INNER JOIN mean exactly the same thing

INNER JOIN = only the matches

Aliases: short names for tables

- orders o = 'I'll call orders just o'
- Less typing, easier to read

```
SELECT o.order_id, c.first_name, o.order_date  
FROM orders o  
JOIN customers c ON o.customer_id = c.customer_id;
```

Joining many tables

- One JOIN for each new table
- Follow the arrows in the ER diagram

```
SELECT o.order_id, p.name, oi.quantity  
FROM order_items oi  
JOIN orders o    ON oi.order_id = o.order_id  
JOIN products p  ON oi.product_id = p.product_id;
```

Forgot ON? Everything gets mixed!

- Without ON, every row is paired with every row
- 15 orders x 10 customers = 150 rows of nonsense
- Too many rows? Check your ON!

LEFT JOIN: keep everyone on the left

- Keeps ALL rows from the left table (the one after FROM)
- No match? The right side becomes NULL
- Now Emma and Lukas are here, with NULL orders

```
SELECT c.first_name, o.order_id  
FROM customers c  
LEFT JOIN orders o ON c.customer_id = o.customer_id;
```

Finding what's missing

- LEFT JOIN + IS NULL = 'who has nothing?'
- Customers who never ordered, products never sold...

```
SELECT c.first_name  
FROM customers c  
LEFT JOIN orders o ON c.customer_id = o.customer_id  
WHERE o.order_id IS NULL;
```

INNER vs LEFT

	INNER JOIN	LEFT JOIN
Keeps	Only matches	Everything from the left table
No match	Row disappears	Row stays, right side is NULL
Use when	You want pairs	You want everyone, or what's missing

Remember

- JOIN puts pieces back together for the answer
- ON says which columns must match
- Aliases (o, c, p) make queries shorter
- INNER JOIN = only matches. LEFT JOIN = keep everyone on the left
- LEFT JOIN + IS NULL finds what's missing

Your turn (1/2)

1. Show every order with the customer's first name, last name and the order status. (15 rows)
2. Show all orders made by Erik. (3 rows)
3. Show all orders from customers in Göteborg, newest first. (3 rows)
4. Show every order item with the product name and category. (23 rows)
5. Which orders contained Shoes? Show order_id and product name. (4 rows)

Your turn (2/2)

6. Show the full receipt for order 10: product name, quantity, unit price and line total. (2 rows)
7. Show which customers have bought a Hoodie Black (first name and order date). (3 rows)
8. Show all customers and their orders, including customers with no orders. (17 rows)
9. Which products have never been sold? (2 rows)
10. Challenge: show customers from Uppsala and every product they bought (first name, product name, quantity). (8 rows)

Coming up tomorrow

**Next week: counting, adding and grouping.
How much money did the webshop make?**